

# Agentic AI Platform

A Plain-English Explanation of the System, Its Components, Its Workflow and Its Architecture

The project, as written on the résumé

*“Increased platform adoption to 1K+ daily users at sub-second latency by architecting a scalable agentic AI system in Python / FastAPI capable of reasoning, planning, tool use, multi-step task orchestration, and safe human-in-the-loop escalation, using LangGraph, MCP, and OpenAI function calling.”*

What is inside this document

| #  | Section                       | What it covers                                         |
|----|-------------------------------|--------------------------------------------------------|
| 1  | The project in simple terms   | the one-paragraph version, and the résumé line decoded |
| 2  | Key words explained           | every technical term in everyday language              |
| 3  | The architecture diagram      | the full picture on one page, then a guided tour of it |
| 4  | The components, one at a time | every block explained end to end: what, why, how       |
| 5  | The workflow, end to end      | the thirteen steps a single request goes through       |
| 6  | A worked example              | one real request followed from question to answer      |
| 7  | When things go wrong          | failures, retries, and why the system stays safe       |
| 8  | The numbers behind the claim  | what 1K+ daily users and sub-second really mean        |
| 9  | Questions you may be asked    | short, honest answers about the design                 |
| 10 | One-page recap                | the whole document in nine lines                       |

# 1. The Project in Simple Terms

## 1.1 The one-paragraph version

This project is an AI assistant that does work, not just an AI that talks. A normal chatbot reads your question and writes back an answer from what it already knows. This system reads your question, works out what you actually want, makes a short plan, then goes and does the steps: it looks things up in the company's real databases, calls internal services, checks policies, and puts the pieces together. If a step is risky, expensive or the system is not confident, it stops and asks a human before acting. Everything runs behind a Python web service so that thousands of requests a day can be handled at once, and the answer starts appearing on the user's screen in less than a second.

A useful way to picture it: imagine hiring a very fast new team member. You do not want someone who only recites facts. You want someone who understands the request, breaks it into steps, knows which internal system to open for each step, does the work in order, notices when something does not add up, and walks over to their manager before doing anything that cannot be undone. This system is the software version of that person, and the design exists to make that behaviour reliable rather than lucky.

## 1.2 The résumé line, decoded phrase by phrase

Each phrase in the résumé bullet is a real engineering decision. This table translates each one before the rest of the document explains it in depth.

| The phrase                                     | What it actually means                                                                                                                                                                                                        |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Increased platform adoption to 1K+ daily users | More than a thousand distinct people use the platform every day. Adoption grew because the assistant became useful enough and fast enough to be worth opening, and reliable enough that people trusted its answers.           |
| at sub-second latency                          | The user does not sit and wait. The first part of the answer appears in under one second, because the reply is streamed word by word instead of being delivered in one block at the end.                                      |
| architecting a scalable ... system             | The design deliberately separates the web layer, the decision layer, the tool layer and the storage layer, so each part can be run on more machines independently as traffic grows.                                           |
| agentic AI system                              | The AI chooses what to do next, not only what to say. It can decide to call a tool, decide to call a second tool based on the first result, and decide when it is finished.                                                   |
| in Python / FastAPI                            | The service is written in Python. FastAPI is the web framework that receives requests over HTTP, checks them, and returns responses. It is asynchronous, which means one server can hold many slow requests at the same time. |
| capable of reasoning                           | Before acting, the system restates the goal in its own words and identifies what information is missing. This one step removes a large share of wrong answers.                                                                |
| planning                                       | It writes an ordered list of sub-steps rather than trying to answer everything in one shot.                                                                                                                                   |
| tool use                                       | It can call real systems: run a database query, search documents, create a ticket, send an email. These are called tools.                                                                                                     |

| The phrase                               | What it actually means                                                                                                                                                                 |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>multi-step task orchestration</b>     | Something has to run the steps in the right order, pass results between them, retry a failed step, and stop a runaway loop. That coordinator is the orchestrator.                      |
| <b>safe human-in-the-loop escalation</b> | For risky actions the system pauses itself, saves exactly where it stopped, and asks a person to approve, edit or reject. Once the person answers, it carries on from that same point. |
| <b>LangGraph</b>                         | The library used to express the agent as a graph of steps with a shared memory, so the flow is explicit, inspectable and resumable instead of hidden inside a loop.                    |
| <b>MCP (Model Context Protocol)</b>      | A standard way to expose tools to an AI. Tools live behind small servers that describe themselves, so new tools can be added without rewriting the agent.                              |
| <b>OpenAI function calling</b>           | The mechanism the model uses to request a tool in a strict, machine-readable form: a tool name plus arguments, instead of free text that would have to be parsed and guessed at.       |

### 1.3 Why an ordinary chatbot was not enough

- A chatbot can only answer from what it was trained on, so it cannot tell a user the status of their own order.
- A single call to a model cannot do a task that needs three lookups in sequence, where step two depends on step one.
- A model asked to produce an action as free text produces something slightly different every time, which is unusable for a real API call.
- Without an explicit pause-and-ask mechanism, the only two options are to let the AI act unsupervised, or to let it do nothing useful. The escalation design is what makes the middle ground possible.

## 2. Key Words Explained

Every term used later in this document, in plain language. Nothing here assumes prior knowledge.

| Term                     | Simple meaning                                                                                                 | Everyday comparison                                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Agent</b>             | Software that decides its own next action to reach a goal, instead of following a fixed script.                | A courier who is given an address, not turn-by-turn instructions. |
| <b>Reasoning</b>         | Working out what the request really means and what is missing before acting.                                   | Reading a work ticket twice before picking up any tools.          |
| <b>Planning</b>          | Turning a goal into an ordered list of smaller steps.                                                          | Writing a shopping list before entering the shop.                 |
| <b>Tool</b>              | A specific capability the AI is allowed to use, such as a database query or sending an email.                  | One app on a phone: it does one job well.                         |
| <b>Tool call</b>         | A structured request from the model naming a tool and its arguments.                                           | A written order slip handed to the kitchen.                       |
| <b>Function calling</b>  | The OpenAI feature that makes the model return that order slip as strict JSON instead of prose.                | A fixed form instead of a handwritten note.                       |
| <b>Schema</b>            | The description of what a tool is called, what it does and which arguments it takes.                           | The label and instructions printed on a switch.                   |
| <b>MCP</b>               | Model Context Protocol: a shared standard for exposing tools so any AI application can discover and call them. | A USB port: one shape, many devices.                              |
| <b>LangGraph</b>         | A library for building an agent as a graph of nodes with shared memory, so the flow is explicit and resumable. | A flowchart that actually runs.                                   |
| <b>Node</b>              | One step in that graph, for example 'plan' or 'call a tool'.                                                   | One box on the flowchart.                                         |
| <b>Edge</b>              | The rule that decides which node runs next.                                                                    | The arrow between two boxes.                                      |
| <b>State</b>             | The shared notebook that every node reads from and writes to during a run.                                     | The case file passed between departments.                         |
| <b>Checkpoint</b>        | A saved copy of the state after each step, so a run can be paused and resumed exactly where it stopped.        | A save point in a game.                                           |
| <b>Human in the loop</b> | A deliberate pause where a person approves, edits or rejects before the system acts.                           | A second signature on a large payment.                            |
| <b>Latency</b>           | How long the user waits.                                                                                       | The gap between ordering and the first plate arriving.            |
| <b>p95 latency</b>       | The wait time that 95 out of 100 requests come in under. It describes the bad days, not the good ones.         | Not the fastest checkout lane, but the one you can rely on.       |

| Term                   | Simple meaning                                                                                | Everyday comparison                                        |
|------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Streaming (SSE)</b> | Sending the answer out word by word as it is produced instead of waiting for the whole reply. | Food served course by course, not all at once.             |
| <b>Guardrail</b>       | A rule enforced in code, outside the model, about what may be sent in or acted on.            | A speed limiter fitted to a vehicle.                       |
| <b>Vector database</b> | A store that finds documents by meaning rather than by exact keyword.                         | A librarian who understands the topic, not just the title. |
| <b>Idempotency</b>     | Designing an action so that running it twice has the same effect as running it once.          | Pressing a lift button twice does not call two lifts.      |

### 3. The Architecture Diagram

The next page holds the whole system on a single sheet, turned sideways so the boxes stay readable. It is worth looking at once before reading anything else, because every later section is a zoom-in on one box of that picture.

#### 3.1 How the picture is laid out

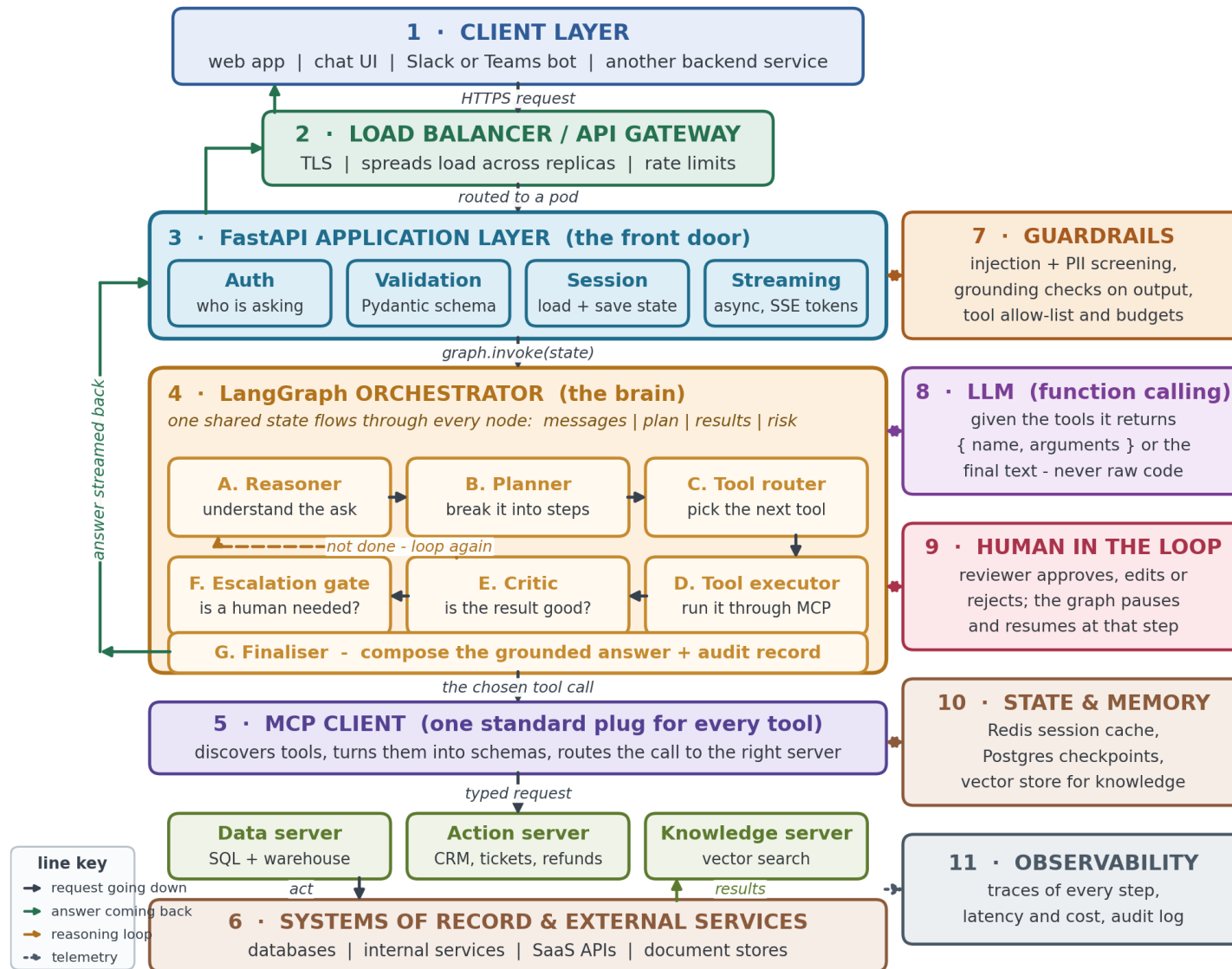
- Read it top to bottom. The user is at the top, the company's real databases and services are at the bottom, and a request travels down and then back up.
- The middle column is the path of the request: front door, brain, tool layer, real systems.
- The right-hand column holds the supporting services that the brain talks to sideways: the safety rules, the language model, the human reviewer, the stores that hold memory, and the monitoring.
- Dark arrows point down and carry the request. The green line on the left carries the finished answer back up to the user.
- The dashed orange arrow inside the brain is the loop: the agent thinking again after seeing a tool result.
- The dotted grey line is telemetry, which is a copy of what happened being written out for monitoring and audit.

Every numbered block is explained in section 4, in the same order.

Figure 1 · The complete system, end to end

## Agentic AI Platform - End-to-End System Architecture

Python / FastAPI | LangGraph | MCP tool layer | OpenAI function calling | human-in-the-loop



Numbers 1 to 11 match the component sections that follow. Nothing in the diagram is decorative: every box is a piece of software that runs, and every line is a real call.

## 3.2 A guided tour of the eleven blocks

This table is the short version. Section 4 then takes each block apart in detail. The third column is the most useful one: it explains why the block exists at all, by describing what would go wrong without it.

| Block                              | What it is                                                                                                                            | What would break without it                                                                                                    |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>1 Client layer</b>              | Whatever the user is typing into: a web app, a chat window, a Slack bot, or another service calling the API.                          | Nothing to serve. It also defines the contract: a question, a user id and a session id.                                        |
| <b>2 Load balancer / gateway</b>   | The traffic officer. It terminates HTTPS and spreads requests across the running copies of the service.                               | One machine takes every request, becomes the single point of failure, and the service cannot grow past it.                     |
| <b>3 FastAPI application layer</b> | The front door of the service: it identifies the caller, validates the request, loads the session, and streams the answer back.       | Bad or unauthenticated input reaches the expensive parts of the system, and the user waits in silence for the full reply.      |
| <b>4 LangGraph orchestrator</b>    | The brain. A graph of small steps sharing one state object: reason, plan, route, execute, criticise, escalate, finalise.              | The agent becomes one unstructured loop that cannot be paused, inspected, resumed or reasoned about when it misbehaves.        |
| <b>5 MCP client</b>                | The universal adapter. It discovers the available tools, describes them to the model, and routes the chosen call to the right server. | Every tool has to be hand-wired into the agent, so adding a capability means editing and redeploying the agent.                |
| <b>6 Systems of record</b>         | The real databases, internal services and third-party APIs where the truth actually lives.                                            | The assistant can only talk about the world in general, never about this user's actual order or ticket.                        |
| <b>7 Guardrails</b>                | Rules enforced in ordinary code: input screening, output checks, per-role tool permissions and budget caps.                           | Safety would depend entirely on the model behaving well, which is not something you can promise anyone.                        |
| <b>8 LLM with function calling</b> | The reasoning engine. Given the conversation and the tool descriptions, it returns either text or a structured tool call.             | No reasoning, no planning, no tool selection. This is the part that decides what to do next.                                   |
| <b>9 Human in the loop</b>         | A reviewer queue and console where a person approves, edits or rejects a proposed action while the run is paused.                     | The only choices left are an assistant that acts unsupervised on risky work, or one that is not allowed to do anything useful. |
| <b>10 State and memory</b>         | Redis for the live session, Postgres for step-by-step checkpoints, and a vector store for long-term knowledge.                        | The agent forgets the previous message, and a paused run can never be resumed because nothing remembers where it stopped.      |
| <b>11 Observability</b>            | Traces, latency and cost metrics, structured logs and an audit record of every escalation.                                            | You cannot tell why an answer was wrong, which step was slow, or prove what happened when someone asks.                        |



## 4. The Components, One at a Time

Each component below follows the same shape: what it is in plain words, what it does in the running system, why it earns its place, and the details that matter in practice.

### 4.1 The client layer

**In simple terms** the surface the person actually touches: a web page, a mobile screen, a Slack message, or another program calling the API directly.

**What it does** it collects the question and sends it over HTTPS together with two identifiers: who the user is, and which conversation this belongs to. It then holds the connection open and paints the answer on screen as the words arrive.

**Why it matters** the session identifier is what turns a series of disconnected questions into a conversation. Without it, 'and what about the other one?' is meaningless.

**Detail worth knowing** the client is deliberately thin. It contains no logic about tools, prompts or policy. Everything that decides behaviour lives on the server, so the rules cannot be tampered with from a browser and can be changed without shipping a new app.

### 4.2 The load balancer and API gateway

**In simple terms** the traffic officer standing in front of several identical copies of the service.

**What it does** it handles the encrypted connection, checks that a copy of the service is healthy before sending work to it, and enforces coarse limits such as how many requests one user may send per minute.

**Why it matters** this is the first half of the answer to 'how does it serve a thousand people a day'. Because the service itself keeps no important data in its own memory, any copy can serve any request, so handling more traffic is a matter of starting more copies.

**Detail worth knowing** rate limiting sits here rather than deeper in the system on purpose. A request that is going to be rejected should be rejected before it costs a single model token.

### 4.3 The FastAPI application layer

FastAPI is the Python web framework that receives the request. Think of it as the front door and the reception desk: it decides whether the visitor may come in, checks their paperwork, finds their file, and later shows them out. It contains four distinct jobs.

#### 4.3.1 Authentication and identity

The request carries a token. The service verifies it and turns it into a known identity: this person, in this team, with this role. That identity is not just for security. It is carried all the way into the tool layer, because a support agent and a finance manager are allowed to trigger different actions. Permission is decided by who is asking, never by what the model asks for.

#### 4.3.2 Request validation

The incoming body is checked against a Pydantic schema, which is a Python class describing exactly what a valid request looks like. A missing field or a wrong type is rejected immediately with a clear error. This is cheap protection: the expensive parts of the system are never even started for a malformed request.

### 4.3.3 Session management

Using the session identifier, the service fetches the recent conversation and the last saved checkpoint. The agent therefore begins with context rather than from nothing. Sessions expire after a set period so that stored context does not grow without limit.

### 4.3.4 Asynchronous handling and streaming

This is the other half of the speed story, and it is worth being precise about it. Most of the time spent in this system is spent waiting: waiting for the model, waiting for a database. Because the endpoints are asynchronous, a single process can hold hundreds of those waits at the same time instead of blocking one request per thread. Meanwhile the answer is streamed back over Server-Sent Events as it is generated, so the user sees the first words almost immediately. The complete answer may take several seconds; the wait before something appears is under one second, and that is the number a user actually feels.

## 4.4 The LangGraph orchestrator: the brain

This is the heart of the project, so it is worth slowing down. An agent needs to do several things in sequence, sometimes repeating a step, sometimes stopping to ask permission. You could write that as a large Python loop with many if-statements, and it would work until the day it misbehaved, at which point nobody could tell what it had done or why.

LangGraph replaces that loop with a graph: a set of small named steps called nodes, connected by rules called edges, all sharing one object called the state. Each node is an ordinary Python function that receives the state, does one job, and returns its changes. Because every step is named and every change to the state is recorded, the run can be watched, paused, resumed and explained after the fact.

### 4.4.1 The state: the shared notebook

The state is the case file that travels from step to step. Every node reads it and writes to it. These are the fields that matter:

| Field               | What it holds                                                           | Who uses it                                                                        |
|---------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>messages</b>     | The conversation so far, including tool results.                        | Every node; it is what gets sent to the model.                                     |
| <b>goal</b>         | The restated request in unambiguous words.                              | Written by the reasoner, read by the planner.                                      |
| <b>plan</b>         | The ordered list of sub-steps, each marked done or outstanding.         | Written by the planner, updated by the critic.                                     |
| <b>tool_results</b> | What each tool actually returned, kept separate from the model's prose. | The critic and the finaliser, so the answer is built from facts, not recollection. |
| <b>confidence</b>   | How well supported the current answer is.                               | The escalation gate.                                                               |
| <b>risk_flags</b>   | Markers such as money involved, irreversible action, sensitive data.    | The escalation gate and the guardrails.                                            |
| <b>attempts</b>     | How many loops have run, and how many times each tool failed.           | The loop limiter, so a run cannot spin forever.                                    |

| Field          | What it holds                                           | Who uses it                                         |
|----------------|---------------------------------------------------------|-----------------------------------------------------|
| human_decision | Approved, edited or rejected, plus the reviewer's note. | Written by the console, read after the run resumes. |

#### 4.4.2 The nodes: the seven steps

Each node does one job and only one job. That is what makes the system debuggable: when the output is wrong, the fault sits in a specific named step.

| Node                     | What it does                                                                                                                        | Why it is a separate step                                                                                                  |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>A Reasoner</b>        | Reads the question and the history and writes down, plainly, what is being asked and what information is missing.                   | Most bad answers come from a misread question. Forcing this into its own step catches that before any work is done.        |
| <b>B Planner</b>         | Turns the goal into an ordered list of sub-steps, for example: find the order, read the refund policy, compare, then decide.        | A plan can be inspected, logged and compared against what actually happened. A hidden plan cannot.                         |
| <b>C Tool router</b>     | Sends the conversation plus the available tool descriptions to the model, and receives back either a tool call or a final answer.   | This is the only place where the model is allowed to choose an action, so every choice passes through one auditable point. |
| <b>D Tool executor</b>   | Validates the arguments, calls the tool through MCP, handles timeouts and retries, and writes the result into the state.            | Execution is ordinary engineering: timeouts, retries and error handling. It should not be entangled with reasoning.        |
| <b>E Critic</b>          | Looks at what came back and judges it: does it answer the sub-step, is it empty, does it contradict something already known?        | This is the difference between an agent that notices a failed lookup and one that confidently invents an answer instead.   |
| <b>F Escalation gate</b> | Scores the situation on confidence, money at stake, reversibility and policy sensitivity, and decides whether a human must approve. | Safety must be a deliberate, configurable decision point, not a hope that the model behaves.                               |
| <b>G Finaliser</b>       | Composes the answer using only the tool results that actually came back, attaches citations, and writes the audit record.           | Keeping composition separate is what allows the answer to be checked against its evidence before it is sent.               |

#### 4.4.3 The edges: how the next step is chosen

An edge is simply the rule that says what runs next. Some are fixed, such as reasoner then planner. The interesting ones are conditional: after the router, the graph goes to the executor if a tool was requested and to the escalation gate if not. After the critic, it loops back to the router if the plan is unfinished, and moves on if it is done. These rules live in ordinary Python and can be read, tested and changed. The model influences the path by what it returns, but it never controls the wiring.

#### 4.4.4 The checkpointer: saving after every step

After each node, LangGraph writes the state to durable storage. That single mechanism buys three things at once. A crashed run can be resumed rather than restarted. A paused run can wait for a human for minutes or hours without holding anything open. And every intermediate step is available afterwards, which is what makes a genuine audit trail possible.

#### 4.4.5 Loop control

An agent that can loop can loop forever. Three limits prevent that: a maximum number of iterations, a token and cost budget per run, and a wall-clock timeout. If any limit is hit, the run does not fail silently. It stops, returns its best partial result with an honest note about what is missing, and, where appropriate, hands the task to a human.

### 4.5 The language model and OpenAI function calling

**In simple terms** the model is the part that thinks. Function calling is the strict form it must use when it wants something done.

Every tool is described to the model as a schema: a name, a sentence about when to use it, and a typed list of arguments. The model does not write code and does not decide what happens next on its own. It returns a small structured object naming a tool and its arguments, which the system is free to validate, reject or execute.

```
the tool, described to the model:
  name:      get_order_status
  description: Look up the current status of a customer order.
  arguments: { order_id: string (required) }

what the model returns when it wants that tool:
{ "name": "get_order_status", "arguments": { "order_id": "A-10423" } }
```

The value of this is that the request is machine-readable by construction. Before the tool runs, the system can check that the arguments match the schema, that this user is allowed to use this tool, and that the values are sane. None of that is possible if the model replies in prose that has to be guessed at with pattern matching.

Model choice is also a performance decision. Simple routing and classification steps go to a small, fast model; only genuinely hard reasoning goes to the larger one. Most steps in a typical run do not need the expensive model, and this routing is a large part of how the latency target is met.

### 4.6 MCP: the tool layer

**In simple terms** MCP is a standard plug shape for tools. Tools live behind small servers that describe what they offer; the agent asks what is available and calls what it needs.

Without a standard, every tool ends up hand-wired into the agent: a function here, a prompt fragment there, a special case in the parsing code. Ten tools later, nobody wants to add the eleventh. With MCP, a tool server declares its own tools and their schemas. The MCP client asks for that list when the service starts, converts it into the function-calling format the model expects, and routes each chosen call to the correct server.

#### What the MCP client does on every call

1. Checks the requested tool actually exists and is currently healthy.
2. Validates the arguments against the tool's declared schema before anything runs.
3. Checks this tool against the allow-list for this user's role, so permission is enforced outside the model.
4. Applies a timeout, and retries with backoff if the failure looks temporary.
5. Returns a typed result, or a clear, structured error the critic can reason about.

The practical payoff is organisational rather than clever: a team that owns a system can publish an MCP server for it, and the assistant gains that capability without the agent's code being touched. New capabilities become a configuration change, which is exactly what made growing the platform to a thousand daily users sustainable.

## 4.7 The tools and the systems of record

A tool is one narrow capability with a clear description. Narrow is deliberate: a tool named 'query\_database' that accepts arbitrary SQL is both hard for a model to use correctly and dangerous. A tool named 'get\_order\_status' that takes one order id is easy to choose correctly and safe to run.

| Example tool                | What it does                                                        | Read or write, and the risk                                                          |
|-----------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>get_order_status</b>     | Returns the current state of one order.                             | Read. Low risk; safe to retry.                                                       |
| <b>search_policies</b>      | Finds the relevant passage in internal policy documents by meaning. | Read. Low risk; grounds the answer in a real source.                                 |
| <b>get_customer_history</b> | Returns recent interactions for one customer.                       | Read, but personal data; access depends on role.                                     |
| <b>create_ticket</b>        | Opens a ticket in the tracking system.                              | Write. Reversible, but must be idempotent so a retry does not create duplicates.     |
| <b>issue_refund</b>         | Moves money.                                                        | Write. Irreversible, so it always goes through human approval above a threshold.     |
| <b>send_email</b>           | Sends a message to a customer.                                      | Write. Cannot be recalled once sent; reviewed when it goes to an external recipient. |

Every write-capable tool is designed to be idempotent, meaning it carries a unique key so that running it twice has the same effect as running it once. Because retries are a normal part of a distributed system, this is what prevents a temporary network hiccup from creating two tickets or two refunds.

## 4.8 State and memory

Three different stores are used because three different jobs are being done, and combining them would make each one worse.

| Store               | What it holds                                                 | Why this store                                                                                                                |
|---------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Redis</b>        | The live session: recent turns and the working context.       | It is held in memory, so reads take about a millisecond. This is on the critical path of every request, so it has to be fast. |
| <b>Postgres</b>     | One checkpoint per completed step, plus the audit records.    | It is durable and queryable. A paused or crashed run can be resumed exactly, and an auditor can be shown what happened.       |
| <b>Vector store</b> | Documents, policies and past resolutions, indexed by meaning. | It answers 'what do we know about this topic' rather than 'find this exact word', which is what grounding an answer requires. |

Memory is trimmed deliberately. Old turns are summarised rather than carried in full, because a longer prompt costs both money and time. Keeping the working context small is one of the least glamorous and most effective latency optimisations in the system.

## 4.9 Human-in-the-loop escalation

This is the component that makes the system deployable in a business at all, and it is the part of the résumé line that the word 'safe' is doing work for.

### 4.9.1 What triggers a pause

Escalation is a rule, not a mood. The gate checks explicit conditions:

- Low confidence: the evidence gathered does not actually support a firm answer.
- Money: the action involves a payment, a refund or a credit above a configured threshold.
- Irreversibility: the action cannot be undone, such as sending an external email or deleting a record.
- Sensitivity: the request touches personal data, legal or medical wording, or a complaint.
- Conflict: two sources disagree, or the plan repeatedly failed to complete.
- Novelty: the request does not resemble anything the system has handled before.

### 4.9.2 How the pause actually works

This is the mechanically interesting part. When the gate decides a human is needed, the graph interrupts itself. The state is already checkpointed, so the run is not sitting in memory waiting: it is saved, and the request thread is released. A task appears in the reviewer queue containing the original request, the plan, the evidence gathered and the exact action being proposed.

The reviewer can approve it, edit it, or reject it with a reason. That decision is written into the state and the graph resumes from the same node it stopped at. It does not start again, it does not lose its work, and it does not re-run the tools it had already run. Because the whole exchange is recorded, there is a permanent record of who approved what and why.

### 4.9.3 Why this design is better than the obvious alternatives

Reviewing everything makes the assistant slower than doing the job by hand, so people stop using it. Reviewing nothing is not acceptable for actions that move money. Targeting review at the small set of genuinely risky decisions keeps the everyday path fast while keeping the dangerous path supervised. The rejections are also useful: they are the clearest signal available about where the rules need to change.

## 4.10 Guardrails

Guardrails are ordinary code that runs outside the model, because a rule the model is asked to follow is a request, while a rule enforced in code is a guarantee.

- On the way in: screening for prompt injection, which is text hidden in a document or a web page that tries to give the agent new instructions, and for personal data that should not be sent onward.
- On the way out: checking that the answer's factual claims trace back to a tool result, and that structured output matches the expected shape.
- Around the tools: an allow-list per role, so the set of things that can possibly happen is bounded no matter what the model asks for.
- Around the run: caps on tokens, loops, spend and elapsed time, so a single request cannot consume the budget of a thousand.

## 4.11 Observability

Every run emits a trace: each node entered, each tool called, how long it took, how many tokens it used and what it cost. Latency is reported as p50, p95 and p99 rather than an average, because an average hides exactly the slow requests that make users abandon a product. Escalations, approvals and rejections are recorded separately as an audit trail. Without this layer, 'sub-second' would be a claim rather than a measurement, and a wrong answer would be a mystery rather than a specific broken step.

## 4.12 How sub-second response was actually achieved

No single trick did it. It is the sum of a number of deliberate choices, each shaving off part of the wait.

| Technique                            | What it means                                                                         | Why it helps                                                                                        |
|--------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Streaming the answer</b>          | Words are sent as they are produced.                                                  | Turns a several-second wait into a sub-second first word, which is the number the user experiences. |
| <b>Asynchronous I/O</b>              | One process holds many waiting requests instead of one per thread.                    | Far more concurrent users per machine, and no queueing behind slow calls.                           |
| <b>Model routing</b>                 | Small fast model for routing and classification, large model only for hard reasoning. | Most steps in a run are simple, and the small model answers them in a fraction of the time.         |
| <b>Parallel tool calls</b>           | Independent lookups run at the same time rather than one after another.               | Three 200 ms lookups cost 200 ms in total instead of 600 ms.                                        |
| <b>Caching</b>                       | Repeated identical lookups and stable prompt prefixes are reused.                     | Removes whole calls from the path for common questions.                                             |
| <b>Trimmed context</b>               | Old turns are summarised rather than resent in full.                                  | Shorter prompts are processed faster and cost less.                                                 |
| <b>Connection pooling</b>            | Database and HTTP connections are kept open and reused.                               | Removes tens of milliseconds of setup from every single call.                                       |
| <b>Tight timeouts with fallbacks</b> | A slow tool is cut off and handled rather than waited on.                             | One slow dependency cannot hold the whole answer hostage.                                           |

## 4.13 Scaling and deployment

The service is packaged in containers and run as several identical replicas behind the load balancer. Because no request-specific data is kept inside a replica's own memory, any replica can serve any request, and more replicas can be started when traffic rises. Session data lives in Redis, checkpoints live in Postgres, and the tools live behind their own MCP servers, each of which can be scaled or restarted on its own. Supporting more than a thousand people a day is therefore not one big machine; it is a design where each part can grow independently and no part holds state that would be lost if it restarted.

## 5. The Workflow, End to End

This section follows one single request all the way through, from the moment a person presses enter to the moment the answer finishes streaming. The two diagrams show the shape of it; the numbered notes underneath explain what is happening inside each step and why it is there.

### How One Request Travels Through the System - Part 1 of 2

arriving, checking, understanding and planning

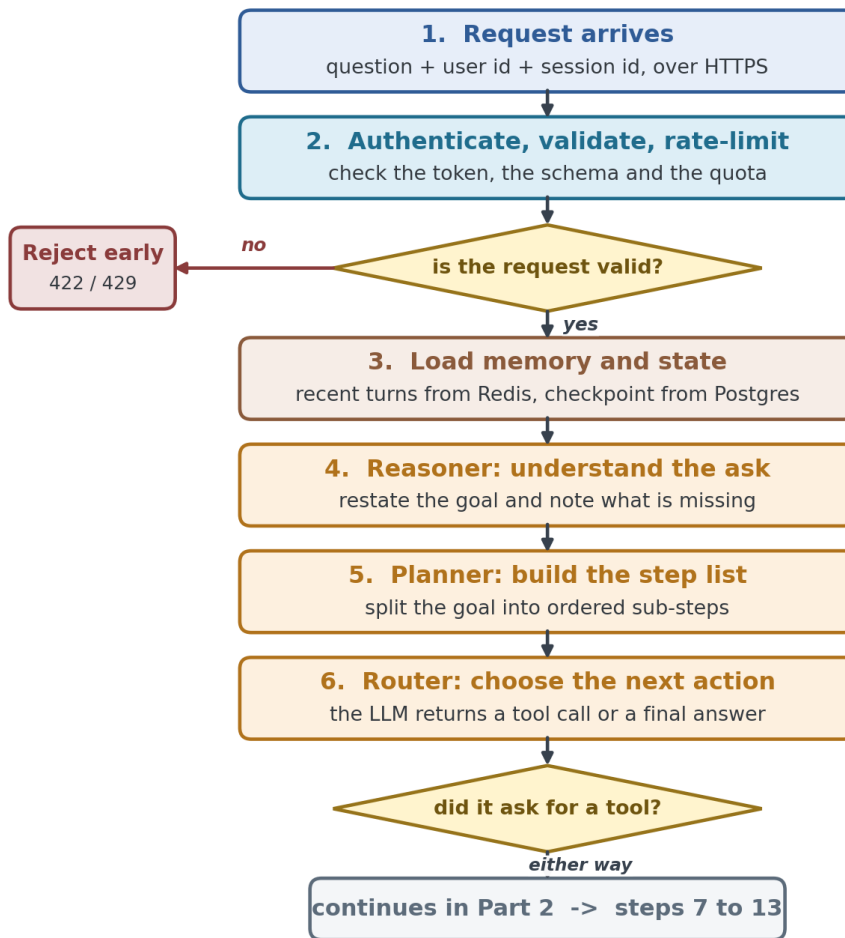


Figure 2 · Steps 1 to 6: arriving, checking, understanding and planning.

### 5.1 Steps 1 to 6 explained

#### Step 1 · The request arrives

The client sends the question together with the user id and the session id over HTTPS. The load balancer picks a healthy copy of the service. Nothing intelligent has happened yet, but the two identifiers attached here determine everything that follows: one decides what the user is allowed to do, the other decides what the assistant remembers.



## **Step 2 · Authenticate, validate, rate-limit**

The token is verified and turned into an identity with a role. The request body is checked against the schema. The user's quota is checked. All three are cheap, and all three happen before any model is called, which is the point: rejecting bad work early is what keeps the cost per useful answer low.

## **Decision · Is the request valid?**

If not, the service returns a clear error straight away and the run ends. No tokens are spent, no tools are touched.

## **Step 3 · Load memory and state**

Redis returns the recent conversation for this session, and Postgres returns the last checkpoint if the conversation is being continued. The agent starts with context instead of amnesia. This is why a follow-up question like 'and the second one?' works at all.

## **Step 4 · The reasoner understands the ask**

The first node of the graph restates the request unambiguously and records what is missing. For example, 'the customer wants to know whether order A-10423 can still be refunded' is a very different instruction from the vague sentence the user actually typed. This is the cheapest error-prevention step in the entire system.

## **Step 5 · The planner builds the step list**

The goal is split into ordered sub-steps: look up the order, read the refund policy, compare the two, then decide. The plan is written into the state so that later steps can tick items off and the critic can tell whether the work is finished.

## **Step 6 · The router chooses the next action**

The conversation so far and the list of available tools are sent to the model. The model returns either a structured tool call or a final answer. This is the only point in the whole system where an action is chosen, which is precisely what makes the behaviour reviewable.

## How One Request Travels Through the System - Part 2 of 2

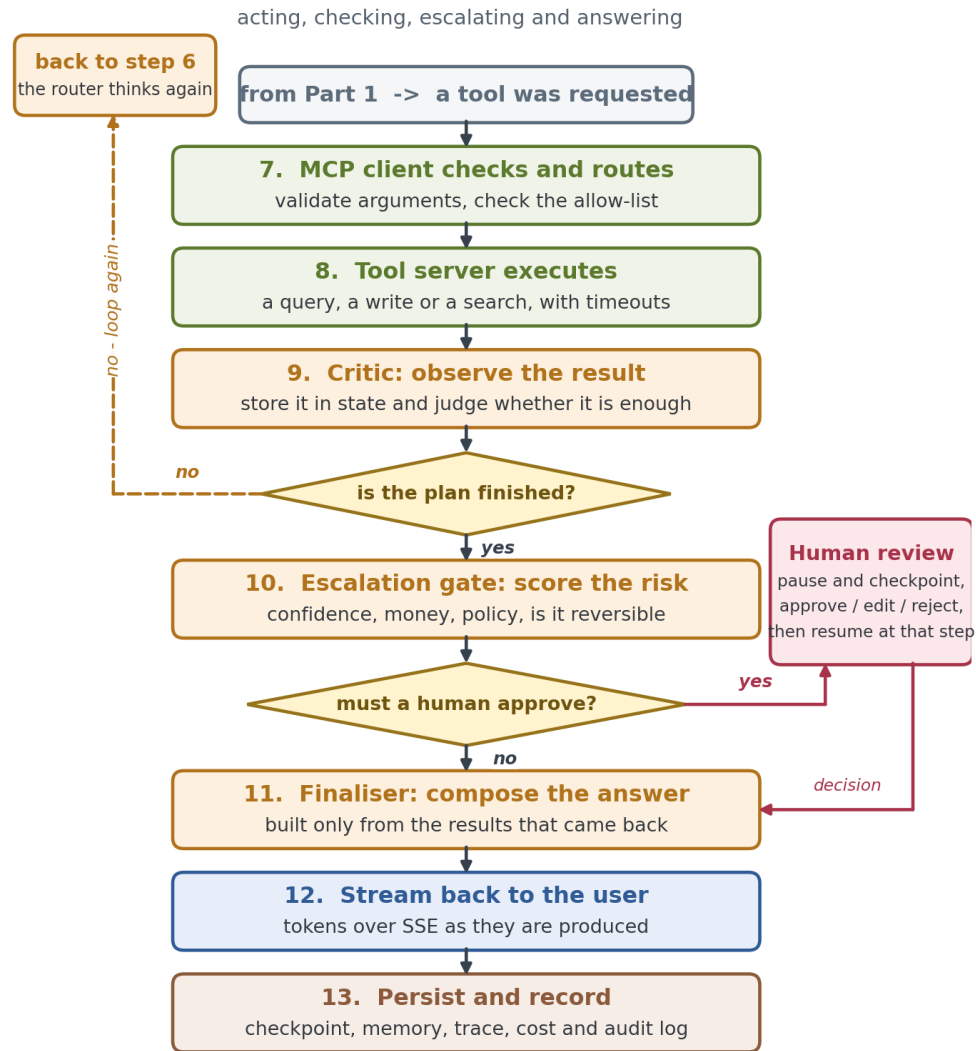


Figure 3 · Steps 7 to 13: acting, checking, escalating and answering.

### 5.2 Steps 7 to 13 explained

#### Step 7 · The MCP client checks and routes the call

The arguments are validated against the tool's schema, the tool is checked against this user's allow-list, and the call is routed to the server that owns it. If the model asked for something that does not exist or is not permitted, it is refused here, in code, before anything runs.

#### Step 8 · The tool server executes

The real work happens: a database read, a document search, a ticket created. Timeouts and retries are applied here, and write actions carry an idempotency key so that a retry cannot duplicate the effect.

### **Step 9 · The critic observes the result**

The result is written into the state and then judged. Is it empty? Does it answer the sub-step? Does it contradict something already known? This is the step that separates an agent that notices a failed lookup from one that quietly invents a plausible answer instead.

### **Decision · Is the plan finished?**

If sub-steps remain, the flow loops back to step 6 and the router chooses the next action, now with the previous result in hand. This loop is the actual 'multi-step orchestration': think, act, observe, think again. It is bounded by the iteration, budget and time limits described earlier, so it cannot run away.

### **Step 10 · The escalation gate scores the risk**

Confidence, money at stake, reversibility and policy sensitivity are scored against configured rules. The result is a straightforward yes or no on whether a person must approve before anything else happens.

### **Decision · Must a human approve?**

If yes, the graph pauses and checkpoints itself, and the task appears in the reviewer queue with the plan, the evidence and the proposed action. The reviewer approves, edits or rejects, that decision is written into the state, and the graph resumes at the very same step. Nothing is re-run and nothing is lost. If no approval is needed, the flow continues directly.

### **Step 11 · The finaliser composes the answer**

The reply is written using only the tool results that actually came back, with citations attached. Composing at the end, from recorded evidence, is what allows the answer to be checked against its sources rather than trusted on faith.

### **Step 12 · The answer streams back**

Tokens are pushed to the client over SSE as they are produced. The user starts reading long before the final word is written.

### **Step 13 · Persist and record**

The final state is checkpointed, the session memory is updated, and the trace, the cost, the latency and the audit record are written. The next question in this conversation will start from here.

## 6. A Worked Example

A customer support agent types: “Can we still refund order A-10423? The customer says it arrived damaged.” Here is what the system does, in order, with rough timings.

| Time    | Step             | What happens                                                                             | Why                                   |
|---------|------------------|------------------------------------------------------------------------------------------|---------------------------------------|
| 0 ms    | Arrive and check | Token verified, body validated, quota checked, session loaded from Redis.                | Cheap checks first.                   |
| 40 ms   | Reason           | Goal restated: decide refund eligibility for order A-10423, damaged on arrival.          | Removes ambiguity before work begins. |
| 120 ms  | Plan             | Three sub-steps: get the order, find the damage policy, compare.                         | Makes the work inspectable.           |
| 200 ms  | Route            | The model asks for get_order_status with order id A-10423.                               | A structured call, not prose.         |
| 260 ms  | Execute          | MCP validates the arguments, the data server returns the order: delivered nine days ago. | The real fact, from the real system.  |
| 300 ms  | Observe          | Critic: useful, but the policy is still needed. Loop back.                               | One sub-step done, one to go.         |
| 380 ms  | Route + execute  | search_policies returns the damage clause: thirty days for damaged goods.                | Grounding, not recollection.          |
| 450 ms  | Observe          | Critic: both facts present and consistent. Plan complete.                                | The loop ends deliberately.           |
| 500 ms  | Score risk       | Refund value is above the auto-approval threshold and moves money.                       | A rule, not a judgement call.         |
| 520 ms  | Escalate         | Run pauses and checkpoints. Reviewer sees the order, the clause and the proposed refund. | Money never moves unsupervised.       |
| -       | Human decides    | The reviewer approves. The decision is written into the state.                           | Recorded for audit.                   |
| +60 ms  | Resume           | The graph continues at the same node; issue_refund runs with an idempotency key.         | No re-running of earlier work.        |
| +200 ms | Answer           | The finaliser writes the reply with both citations; tokens stream to the screen.         | Checkable against its sources.        |

Two things are worth noticing. First, the user saw words appear on screen in well under a second, even though the whole task involved two tool calls and a human approval. Second, at no point did the model decide it was allowed to move money. It proposed; the rules routed; a person approved.

## 7. When Things Go Wrong

Most of the engineering in a system like this is not about the day everything works. These are the failures that were designed for.

| What goes wrong                               | What the system does                                                                                    | Why that is the right response                                                  |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| A tool times out                              | Retries with backoff, then reports the failure into the state.                                          | Most timeouts are temporary; a retried read is harmless and usually succeeds.   |
| A tool keeps failing                          | The critic marks the sub-step as blocked and the run finishes with an honest partial answer.            | A clear 'I could not check this' is far more useful than a confident guess.     |
| The model asks for a tool that does not exist | Rejected by the MCP client before execution, and the error is fed back so the model can correct itself. | Validation in code, not trust in the model.                                     |
| The model loops without progressing           | The iteration and budget limits stop the run and it escalates.                                          | Bounded cost and bounded time, always.                                          |
| A document contains hidden instructions       | Input screening strips or flags it, and tools stay bounded by the role allow-list.                      | Even a successful injection cannot exceed the permissions of the person asking. |
| The service restarts mid-run                  | The last checkpoint is reloaded and the run resumes.                                                    | Work already paid for is not thrown away.                                       |
| A reviewer rejects an action                  | The rejection and its reason are recorded, and the user is told plainly.                                | Rejections are the most valuable signal for improving the rules.                |
| Traffic spikes                                | The load balancer spreads it across more replicas; rate limits protect the shared dependencies.         | The system degrades gracefully instead of falling over.                         |

## 8. The Numbers Behind the Claim

### 8.1 What 'sub-second' means precisely

It means time to first token: the gap between the user pressing enter and the first words appearing. That is measured at the p95 level, meaning ninety-five out of a hundred requests come in under it, not as an average that would hide the slow ones. The complete answer, especially one involving several tool calls and a human approval, naturally takes longer. The claim is about responsiveness, which is what a user actually perceives, and it is honest to state it that way.

### 8.2 What '1K+ daily users' means precisely

More than a thousand distinct people use the platform on a normal day. That number is a consequence of the design rather than a separate achievement: the service holds no state of its own so it can be replicated freely, asynchronous handling lets each replica hold many waiting requests at once, and the session and checkpoint stores are separate systems that scale on their own.

### 8.3 Why adoption grew

- The assistant could answer questions about the user's own data, which a general chatbot cannot do.
- It was fast enough to be worth opening instead of doing the lookup by hand.
- Its answers cited the source they came from, so they could be checked rather than trusted blindly.
- It never quietly did anything risky, which is what made a business willing to leave it switched on.

---

## 9. Questions You May Be Asked

### Why LangGraph instead of writing the loop yourself?

Because the loop is the easy part; pausing, resuming, inspecting and auditing it is the hard part. LangGraph provides an explicit graph with a shared state and a checkpoint after every step, which is exactly what human-in-the-loop escalation needs. A hand-written loop would have had to grow all of that anyway, less carefully.

### Why MCP rather than just defining functions in the code?

Because tools multiply. MCP makes a tool a self-describing service rather than a hard-wired function, so another team can publish one and the assistant gains the capability without the agent being modified. It turns adding a capability into a configuration change instead of a release.

### How is it safe if the model is unpredictable?

Safety does not depend on the model. The model can only propose a tool call. Whether that call is permitted is decided by the allow-list for the user's role, whether the arguments are acceptable is decided by schema validation, and whether a risky action proceeds is decided by the escalation rules. All three are ordinary code, outside the model.

### How do you get sub-second responses out of a system that calls tools?

By streaming the answer so the first token arrives quickly, by routing simple steps to a small fast model, by running independent tool calls in parallel, by caching repeated work, by keeping the prompt short, and by reusing connections. No single one of those is decisive; together they are.

### What stops it from looping forever or spending without limit?

A maximum iteration count, a token and cost budget, and a wall-clock timeout on every run. When a limit is reached the run stops and returns an honest partial answer rather than failing silently.

### What was the hardest part?

Deciding what to escalate. Escalating too much makes the assistant slower than doing the work by hand and people stop using it; escalating too little is unacceptable for anything that moves money or cannot be undone. Getting that boundary right, and making it configurable rather than hard-coded, mattered more to adoption than any individual piece of technology.

---

## 10. One-Page Recap

- The system is an AI that does work rather than one that only talks: it reasons, plans, uses real tools, and knows when to stop and ask.
- FastAPI is the front door: it authenticates, validates, manages the session and streams the answer back.
- LangGraph is the brain: seven small named steps sharing one state object, with a checkpoint saved after each one.
- OpenAI function calling is how the model asks for an action in a strict, checkable form instead of prose.

- MCP is the standard plug for tools, so capabilities can be added without touching the agent's code.
- Human-in-the-loop escalation pauses the run, saves it, gets a decision, and resumes at the same step.
- Guardrails, budgets and allow-lists live in ordinary code, so safety does not rest on the model behaving.
- Sub-second means time to first token at p95, achieved through streaming, async I/O, model routing, parallel calls and caching.
- More than a thousand daily users is supported by stateless replicas behind a load balancer, with session and checkpoint storage kept separate.